MARS: Reducing MSHR Admission Stalls with Selective Miss Forwarding

***Abstract*—Miss latency dominates performance loss in memory-intensive workloads. Prior work resolves it along three axes: prefetching shifts when a miss issues, replacement reshapes which lines a level retains, and off-chip prediction overlaps the DRAM probe with on-chip lookup. Each family moves a miss in time, in space, or in destination, however, none addresses the cycles in which a cache level cannot admit a newly detected miss because its Miss Handling Concurrency is already saturated. We call these admission stall cycles, and we show they concentrate at the first-level cache, where the unfiltered demand stream meets the tightest admission capacity while lower-level resources are underutilized. We propose MARS, a selective, adaptive, and lightweight forwarding framework that hands qualifying misses to the next level so its underused capacity absorbs them. The decision reads short- and long-window signals of hit activity and pure-miss ratio drawn from the concurrent memory access model, and a three-gate model triggers only when throughput drops and ratio rises compared to the level's own baseline. The mechanism lifts a level's effective miss handling concurrency past its structural bound, preserves baseline ordering and consistency, and collaborates with different prefetchers and replacement policies that the system uses. Across SPEC CPU traces and LLM inference traces, MARS reduces admission stall cycles and delivers a 4.3% geomean speedup, at 92 bits of storage at L1 and L2.**

I. Introduction

The widening gap between processor speed and main memory latency [1] has made memory access a dominant bottleneck in modern memory-intensive workloads [2]. To mitigate the long latency of off-chip main-memory accesses, modern processors use multi-level cache hierarchies to exploit data locality. When a requested block is absent from a cache level, the resulting miss propagates downward through the hierarchy until the data is found or fetched from DRAM. Beyond exploiting locality, processors further improve data-access performance through concurrency support, allowing independent memory accesses to overlap rather than being serialized [3], [4]. In particular, non-blocking caches [5], [6] enable multiple outstanding misses to be serviced in parallel, with Miss Status Holding Registers (MSHRs) tracking these in-flight misses until the requested data returns. However, finite MSHR capacity at each cache level bounds the number of concurrent misses supported by hardware [7]. Once all MSHR entries at a level are occupied, the level cannot admit newly detected misses until an entry is released, limiting overall performance.

To improve memory-system performance, prior work has mitigated the performance impact of cache misses from three complementary directions. *Hardware prefetching* anticipates future cache misses and issues speculative requests before the corresponding demand accesses occur, shifting miss handling earlier to hide memory latency [8]–[14]. *Replacement policies* improve cache residency by predicting which cache lines are likely to be reused and evicting lines with low predicted value, thereby reducing future misses [15]–[19]. *Off-chip predictors* identify demand requests likely to miss across the cache hierarchy and initiate parallel off-chip accesses [20]–[22], overlapping costly DRAM access with on-chip lookups across cache levels and masking off-chip latency behind cache-hierarchy traversal.

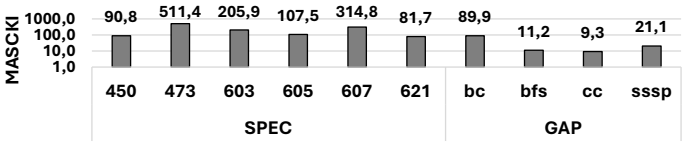

Fig. 1: MSHR Admission Stall Cycles Per Kilo instructions across different SPEC and GAP traces.

However, hardware prefetching, replacement policies, and off-chip predictors mainly reduce the number of cache misses or hide miss latency; they do not address the performance loss caused by *MSHR Admission Stall Cycles*, defined as cycles in which a detected miss waits because all MSHR entries at that cache level are occupied. Figure 1 quantifies this bottleneck using MSHR Admission Stall Cycles per Kilo Instructions (MASCKI), which measures the number of cycles per thousand instructions during which L1 cannot admit a detected miss because all MSHR entries are occupied. Under LRU replacement without prefetching, SPEC workloads exhibit between 81.7 and 511.4 MASCKI, while GAP workloads range from 9.3 to 89.9 MASCKI. At 511.4 MASCKI, the accumulated stall time is comparable to several complete cache-hierarchy traversals, during which a known miss remains queued at L1 before it can initiate request the next cache level. Unlike conventional miss latency, these stalls cannot be mitigated by prefetching, replacement policies, or off-chip predictors, since no new miss can be admitted once all MSHR entries are occupied.

A direct solution is to enlarge the MSHR structure [23], which increases the local miss-handling capacity. However, this approach does not exploit the imbalance across the cache hierarchy: upper-level MSHRs may be saturated while lower-level MSHR resources remain available. This observation motivates a different approach: forwarding selected misses to lower cache levels with available miss-handling capacity, thereby utilizing available hardware-supported concurrency instead of stalling while waiting for an upper-level MSHR entry to be released.

To this end, we propose MARS, a selective miss forwarding framework that reduces MSHR admission stalls by exploiting available miss-handling capacity across cache levels. First,

MARS allows selected misses to bypass local MSHR allocation at an upper cache level and be handled by the next level, mitigating stalls caused by waiting for an upper-level MSHR entry to be released. Second, MARS uses lightweight concurrency-aware runtime signals to identify when local cache-level handling provides limited benefit and forwarding becomes more effective. This enables MARS to forward misses selectively rather than blindly bypassing the current level. Finally, MARS improves effective miss-handling concurrency with low hardware overhead and remains complementary to existing prefetching, replacement, and off-chip prediction mechanisms.

The performance of MARS is extensively evaluated across memory-intensive workloads from SPEC [24], [25] and GAP [26], under multiple prefetchers, replacement policies and core configurations. The results demonstrate that MARS, by targeting both MSHR Admission stall cycles and degrading performance miss, consistently outperforms state-of-the-art (SOTA) off-chip prediction schemes, including TTP [21], HMP [20] and Hermes [21]. Notably, in the 1-core configuration, MARS achieves 42.1% performance improvement on SPEC workloads, outperforming these SOTAs by 5.1%, 4.9%, and 2.1%, while reducing L1 MSHR Admission stalls by 30.6% and pure misses by 34.4%. Moreover, MARS achieves 22.3% improvement on GAP workloads over the base line configuration.

This paper makes the following contributions:

- We introduce and characterize MSHR Admission stall as a first-level structural bottleneck that no prior families actively addresses.
- We propose MARS, a selective, adaptive and lightweight forwarding framework that resolves MSHR Admission stalls by redirecting qualifying misses so the next level's MSHR, raising Miss Handling Concurrency past its structural bound while preserving consistency.
- We derive MARS's three-gate forwarding model from short and long window using hit and miss performance signals, without storing per-miss state at any skipped level.
- We extensively evaluate MARS across SPEC CPU and GAP workloads under multiple prefetching and replacement policy configurations, consistently outperforming Hermes, HMP, and TTP across both workloads.

II. Background

A. Concurrent Memory Access Model

Modern memory systems support concurrent data accesses, allowing independent accesses to overlap rather than execute serially. As a result, cache hits and outstanding misses may occur in the same cycle. The Concurrent Average Memory Access Time (C-AMAT) model [4] captures the impact of data access concurrency by classifying memory active cycles ω at each cache level into three categories: a pure hit cycle h, where only cache hits are serviced without overlapping miss activity; an overlap cycle x, where hit service overlaps with outstanding miss activity; a pure miss cycle m, where outstanding miss activity exists but no hit service occurs and an idle cycle e where no memory activity is present.

Based on these categories, two performance factors, φ and κ, are introduced to characterize cache level concurrency behavior [27], [28]. φ measures the fraction of memory active cycles that include useful hit service, as shown in Equation 1. A high φ indicates that the cache level actively provides useful hit service during memory active cycles, reflecting data locality benefit at this level. κ measures the fraction of miss related active cycles that appear as exposed pure miss behavior rather than being hidden by hit service, as shown in Equation 2. A high κ indicates that more miss activity appears as exposed pure miss cycles, suggesting limited hit service overlap and weaker ability to hide miss latency.

$$\varphi = \frac{h + x}{\omega} \qquad (1) \qquad\qquad \kappa = \frac{m}{m + x} \qquad (2)$$

Takeaway 1: φ and κ provide a compact way to evaluate cache performance by jointly capturing data locality benefit and the effectiveness of hit and miss overlap.

B. MSHR as the Miss-Handling Resource

On a miss, the cache checks whether the missed block is already tracked by an existing MSHR entry. If so, the new request merges with that entry. Otherwise, the miss becomes a new primary miss and requires a free MSHR entry to track it until the data returns from the next level. The number of MSHR entries, denoted as $\mathrm{MSHR}_{\mathrm{size}}$, limits how many primary misses can be tracked concurrently at a cache level. The number of occupied MSHR entries, denoted as $\mathrm{MSHR}_{\mathrm{occ}}$, reflects the current miss handling concurrency supported by hardware at that level. MSHR entries are released only after the corresponding misses are serviced by the next level. Therefore, longer downstream service latency keeps entries occupied for more cycles and increases the chance that new primary misses cannot be admitted.

When $\mathrm{MSHR}_{\mathrm{occ}}$ reaches $\mathrm{MSHR}_{\mathrm{size}}$, the cache level has no free entry for a new primary miss, so the miss cannot be admitted into the local miss handling pipeline until an entry is released. We refer to this condition as an MSHR admission stall. During this stall, already admitted misses may continue to be serviced, but the incoming primary miss must wait, delaying memory access and degrading overall performance. MSHR admission stalls are asymmetric across the cache hierarchy. They appear more frequently at upper cache levels, especially L1, because L1 first receives the demand stream, has limited MSHR capacity, and must keep entries occupied until misses are serviced by lower cache levels or main memory. Since L1 is closest to the core, stalls at this level directly delay demand requests and are therefore performance critical. In contrast, lower cache levels receive a filtered stream of misses and may still have available miss handling capacity when the upper level is saturated. This imbalance motivates MARS to forward selected misses to lower cache levels instead of waiting for an upper level MSHR entry to be released.

Takeaway 2: Upper-level MSHR saturation degrades performance and creates an opportunity to forward selected misses to lower levels with available miss-handling capacity.

C. Existing Cache Performance Optimizations

Existing cache optimizations improve cache hierarchy performance from three complementary directions, each targeting a different aspect of cache-miss behavior.

Hardware Prefetching. Hardware prefetchers [8]–[14] predict future memory accesses and issue speculative requests before the corresponding demand accesses occur. SPP [10] compresses stride history into a signature and walks a speculative delta chain until confidence drops. Zion [12] runs five independent temporal-spatial modules with feedback-directed throttling that lags by one epoch after accuracy collapses. Bingo [9] replays a spatial bitmap of co-accessed lines on a trigger match, issuing many prefetch requests simultaneously. Berti [8] selects deltas whose observed latency meets the current LLC round-trip time, discarding stale mappings after a pattern shift.

Replacement Policies. Cache replacement policies [15]–[19] improve cache residency by selecting which cache lines should be retained or evicted. LRU evicts the least recently used line and serves as a simple recency-based baseline. SHiP++ [17] predicts reuse behavior using instruction-based signatures and adjusts insertion priority based on historical reuse. Hawkeye [18] approximates Belady's optimal policy over a sampled history window to classify cache lines as cache-friendly or cache-averse. CARE [19] incorporates concurrency awareness by estimating the performance impact of cache misses through pure-miss contribution and using it to guide eviction decisions.

Off-chip Predictor. Off-chip access predictors [20]–[22] identify load requests that are likely to miss across the cache hierarchy and issue early DRAM requests to overlap off-chip latency with on-chip cache traversal. TTP [21] tracks per-instruction hit and miss behavior to predict long-latency misses and trigger speculative memory accesses. HMP [20] uses IP-indexed prediction tables to classify load behavior and initiate early off-chip requests for predicted cache misses. Hermes [21] improves prediction accuracy by using a perceptron-based predictor that combines multiple load-related features.

Overall, these mechanisms improve memory-system performance by reducing cache misses, improving cache residency, or hiding miss latency. However, they do not directly address MSHR admission stalls, where a newly detected miss cannot allocate an MSHR entry at the current cache level.

Takeaway 3: Existing mechanisms mainly focus on reducing cache misses or miss costs, but do not directly address the performance impact of MSHR admission stalls.

Before I move on to the next part, please carefully review my revisions and think about why I made these changes. Please take more ownership of the next revision and make sure Section 3 is carefully revised to a near submission-ready level before sending it back. With the help of current AI and writing tools, basic scientific writing and consistency should be improved before my review, so I need to see more careful checking and stronger ownership from your side.

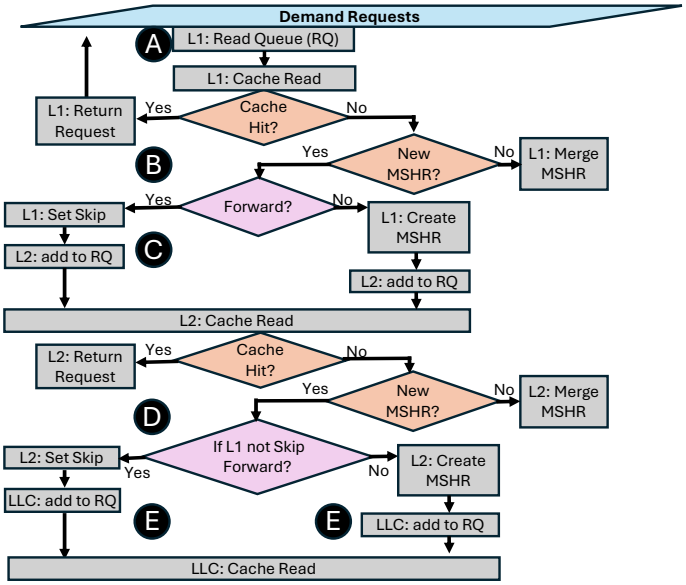

Fig. 2: MARS downward forwarding path: a per-level skip-bit replaces MSHR allocation and routes the miss to the next level's read queue.

III. MARS: Reducing MSHR Admission Stalls with Selective Miss Forwarding

Traditionally, MSHR is used to track and propagate the miss downwards the cache hierarchy until it reaches DRAM, to service levels upwards. We propose MARS, an MSHR Admission stall Reduction via Selective Miss forwarding framework operating within the cache. To make a decision, it evaluates incoming misses against two windowed performance signals (φ and κ) from the concurrent memory access model. When MARS forwards the request, MSHR allocation at a level is omitted and lower-level receives a marked demand request along the standard miss path. The signal indicates no upper-level MSHR exists, and requires the receiving lower-level to allocate MSHR to track both levels simultaneously. As a result, on the data return, the lower-level acts as a miss tracking anchor routing the return directly to upper-level's requester while filling the skipped level to keep the cache content consistent. The framework restricts forwarding to a single non-adjacent skip per-in-flight request, so the lower-level's MSHR absorbs the miss and decisions are kept localized to align with the standard cache hierarchy.

The remainder of this section details the downward forward propagation (Section III-A), the three-gate model in MARS controlling decisions (Section III-B), the upward return path (Section III-D), Miss Handling Concurrency (Section III-C) and the storage overhead (Section III-E).

A. Downwards Forwarding

Figure 2 illustrates the MARS flow through the cache hierarchy. A demand (A) enters the L1 read queue (RQ) along the standard path. On a new miss, MARS three-gate model is triggered to make a forwarding decision (B). When selected, instead of MSHR allocation, the request receives a level-skip bit

$L1_{skip}$ (C), indicating no MSHR allocation, and propagates directly into the RQ of L2. When chosen to keep, L1 allocates the MSHR entry and the request proceeds to L2 through the conventional path.

At L2, the L1 miss follows the standard path through the RQ for a cache lookup. On a hit, if the request carried the $L1_{skip}$ bit, L2 returns the demand directly to the upper-level's requester without filling it. For a new L2 miss, (D), the $L1_{skip}$ bit is checked, if set, the MSHR is allocated to anchor both its own and the upwards-stream miss. When L1 was not skipped, MARS is triggered and either sets $L2_{skip}$ bit to forward the request without MSHR allocation, or allocates the L2 MSHR so it anchors the miss. In both cases, (E), the demand propagates to LLC's RQ.

At LLC, the L2 miss is read from the RQ. On a new miss, if $L2_{skip}$ is not set, the L2 MSHR already handles its own miss and potentially the L1 miss as well (if L1 was skipped). We always set the LLC_{skip} so the request is routed directly to DRAM without MSHR allocation. In cases when $L2_{skip}$ was set, LLC creates MSHR entry to avoid creation of an upstream handling gap.

When two consecutive levels do not track a miss, the lower-level is responsible to resolve the full upwards return chain, as routing complexity compounds with each additional skip.

B. Three-gate Forwarding Decision Model

MARS maintains four cycle counters at each cache level: pure-hits h, pure-misses m, overlaps x and a window counter, used in short and long history windows. The long window acts as a baseline against recent performance signals to compare them. We utilize 512 cycles for immediate demand behavior and 16384 cycles for steady-state history. When a window limit is reached, all counters are halved rather than reset, offering a decaying average where recent cycles carry greater weight than old history.

Counter-halving is also a natural adaptive throttling mechanism, which reduces the frequency of $\mathrm{MSHR}_{\mathrm{skip}}$ under sustained pressure by narrowing the short-long signal gap. As a result, the model triggers less frequently at the level, permitting its lower-level to make its own forwarding decisions more often to alleviate its own MSHR pressure. Using the counters MARS computes the concurrent memory access model ratios φ and κ defined in Equations 1 and 2: φ measures the fraction of active cycles including both beneficial and neutral work (h and x), while κ measures the fraction of pure-miss activity (m) stalling the hierarchy [27].

MARS forwarding decisions are made using a three-gate model at every new miss occurred at the L1 or L2 (Figure 3). Only when all three gates satisfy their conditions ($Gate_1 \wedge Gate_2 \wedge Gate_3$) the level forwards a miss to the lower-level. $Gate_1$ confirms the level is under the active miss pressure - without it, there is nothing to relieve. When satisfied, the model uses short and long-term φ in $Gate_2$, to detect declining hit activity relative to the level's own history. Forwarding a miss prevents it from occupying an MSHR entry at the level, which frees capacity allowing potential overlap or pure-miss cycles to become pure-hits or idle cycles instead, pushing φ toward its baseline. However, $Gate_2$ alone cannot tell whether the φ drop comes from fewer pure-hits or from fewer overlap cycles. $Gate_3$ resolves this since rising κ_s would indicate more pure-misses, resulting in less overlap. The AND condition prevents forwarding when φ drops but κ is unchanged, or when κ rises but φ activity is adequate. Comparing each ratio against its long-window value sets a per-level reference, so the gates activate only when the level deviates from its own history.

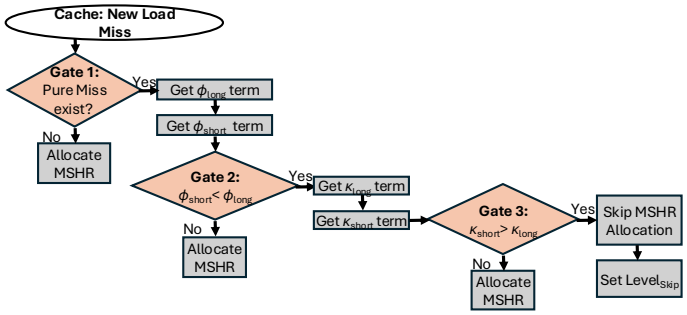

Fig. 3: Three-gate model making forwarding decisions using short and long windows, considering κ, φ.

TABLE I: Gate-pair states when Gate 1 is satisfied ($m_s > 0$).

$Gate_2$	$Gate_3$	**Interpretation**	**Action**
0	0	Level at baseline: no miss pressure	Keep
1	0	φ declining: miss overlap x stable	Keep
0	1	κ rising: hit activity sustained	Keep
1	1	φ declining AND κ rising	**Forward**

Table I summarizes the interpretation of each condition used, $Gate_2$ and $Gate_3$, assuming $Gate_1$ is already satisfied. When neither activates, the level operates within its baseline and no forwarding is needed. Gate 2 alone signals a hit-side disturbance without rising a pure-miss ratio, so the level is not stalling and MARS keeps the miss. Gate 3 alone signals a rising pure-miss ratio that sustained hit could be absorbing, so forwarding here disrupts a stable flow and blocks a lower-level decision with greater latency benefit. Only $(G_2{=}1 \wedge G_3{=}1)$ is able to confirm declining hits and losing overlap simultaneously which is the state when MARS forwards the request.

During an MSHR Admission stall, outstanding misses saturate MSHR and blocks new allocations. When a miss is unable to be admitted, pure-miss cycles rise immediately, satisfying Gate 1 ($m_s > 0$) within the short window. The pure-miss traffic simultaneously triggers reduction of φ_s below the long-window baseline (since overlap cycles and hit cycles are absent) and quickly elevates κ_s above the baseline, triggering all three gates, allowing to forward the MSHR Admission stall miss.

C. Miss Handling Concurrency

L1 and L2 each run the three-gate model independently with private counters at each level. L1 observes its own unfiltered h, m and x cycles and L2 sees any L1 admitted miss or forwarded cycles without cross-level state sharing. The LLC does not run the gate model, since the choice of L2 indirectly affects it and LLC's traffic tends to be spread across where non-overlapping

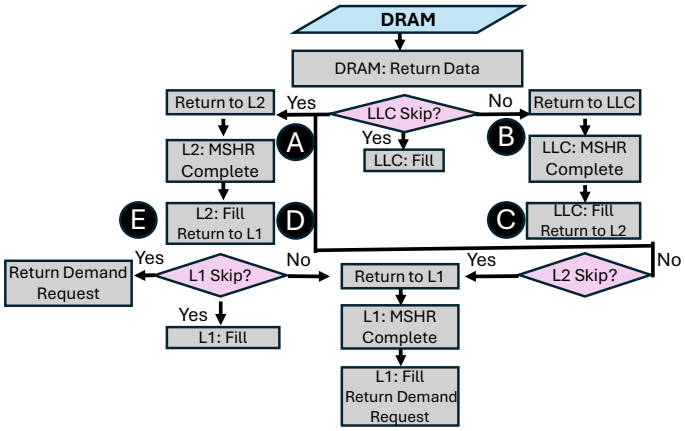

Fig. 4: Upwards return: direct route to requester; decoupled fill at skipped levels.

pure-misses dominate which persistently makes high κ and low φ, making the gate's state not bring corresponding benefits.

Because a forwarded miss counts towards $\mathrm{MSHR}_{\mathrm{skip}}$ rather than $\mathrm{MSHR}_{\mathrm{occ}}$, we define Miss Handling Concurrency (Equation 3) as the number of misses originating at the level that are currently in-flight ($\mathrm{MSHR}_{\mathrm{occ}}$) plus the misses forwarded to a lower-level that are also in-flight ($\mathrm{MSHR}_{\mathrm{skip}}$).

$$\mathrm{MHC} = \mathrm{MSHR}_{\mathrm{occ}} + \mathrm{MSHR}_{\mathrm{skip}} \tag{3}$$

In the baseline, MHC is bound to MHC_{Ω} of the first level that accepts the demand request. Utilizing MARS forwarding raises the peak MHC via a lower-level's MSHR. The upper bound MHC_{Ω} (Equation 4) is reached when the first level's in-flight misses fully occupy its MSHR plus the hit-relaxed number of forwarded misses the next level sustains, $\mathrm{MSHR}_{(\mathrm{L+1,size})}/M_{\mathrm{L+1}}$, where L denotes the first cache level accepting the demand request and $M_{\mathrm{L+1}}$ the miss ratio of the lower-level $L{+}1$.

$$\mathrm{MHC}_{\Omega} = \mathrm{MSHR}_{\mathrm{L,size}} + \frac{\mathrm{MSHR}_{(\mathrm{L+1,size})}}{M_{\mathrm{L+1}}} \tag{4}$$

D. Upwards Forwarding Return

Figure 4 depicts the data return path, the 3-b skip tag determines which levels were skipped on the forward path (Section III-A). DRAM checks the LLC_{skip} bit, if set (A), it routes the return directly to L2, while concurrently filling the skipped LLC via its replacement policy, otherwise, (B), LLC completes its MSHR normally, gets filled, and returns to the upper-level (C). Any returned request to L2, during the return would have the $L1_{skip}$ checked, (D), if set, it returns directly to the requester (E), filling L1 in parallel, otherwise returns to L1 where MSHR completes, level fills, and data returns to the requester. Furthermore, when data returns to LLC from the DRAM, before turning to L2, (C) it would check $L2_{skip}$ and direct the return to L1 while filling L2.

The data return path routes directly from the resolving level to the requester, reducing the return latency without touching the skipped levels. The fill at each skipped level overlaps with the lower-level's data return, maintaining cache content consistency. The skipped level fill enables replacement policies to retain the line and serve repeated demands as cache hits, while forwarding reduces MSHR Admission stalls. On the return, skipped levels are untouched and data routes directly to the requester, reducing the return latency. Likewise, the skipped level fill enables replacement policies to retain the line and serve repeated demands as cache hits.

TABLE II: Storage overhead of MARS

Structure	Field	Ent.	#Bits
Cycle Counter	short (9 b): h, m, x	3	27
	long (14 b): h, m, x	3	42
Window Counter	short (9 b) + long (14 b)	–	23
Request tag	level-skip (3 b)	in-flight	3
		Cache Level total:	**92 b**

Consistency. The return path is consistent with the baseline cache. Three cases cover every forwarded miss:

1) At the originating forwarding level. **Downwards** on a repeated demand if the repeated demand creates MSHR at the skipped level the request propagates to the lower-level. It will be merged into the existing MSHR, which will clear the skip bit. **Upward**, when data returns, presence of the bit determines whether an upper-level awaits a return, based on this, the level routes to the upper-level or its requester accordingly.

2) At an intermediate level whose skip bit is set: the level holds no MSHR entry for the miss. **Downwards** path there cannot be any new repeating request to the level, because it would be merged during the upper-level's MSHR anchor (Section III-A) creating no new downward requests. **Upward** propagation will not interact with the skipped level, as MSHR entry cannot exist here, because it would mean the upper-level has already been served.

3) At a level whose skip bit is clear, the level holds the standard MSHR entry. It completes the normal fill and is identical to the baseline.

E. Storage Overhead

At each gated level (L1 and L2), MARS maintains two sampling windows: a short (9-bit) and a long (14-bit). Each tracks three cycle counters (h, m, x) and window counter itself. The per-gated-level storage requirement is $4 \times (9 + 14) = 92$ bits. The LLC runs no gate model and stores no gate counters, therefore, total cache storage is 23 bytes. Each in-flight miss carries a separate 3-bit skip tag on the request packet. The gate logic reuses the existing miss-path control and adds no MSHR entry at any skipped level. Table II summarizes the per-gated-level breakdown.

IV. Methodology

Simulated system. We evaluate MARS using a modified ChampSim simulator [29], used in the 2nd and 3rd Data Prefetching Championships [30], [31]. We simulate the Intel Alder Lake (GLC) [32] multi-core processor as the default microarchitecture in 1-core to 16-core configurations. Table III(A) lists the parameters. We also simulate the Apple A14 Firestorm

TABLE III: Simulated system parameters.

(A) (Default) Intel Alder Lake (GLC)	
Core	1/4/8/16 cores, 6-wide fetch/execute/commit, 512-entry ROB, 128/72-entry LQ/SQ, perceptron branch predictor
L1/L2 Cache	Private, 48KB/1.25MB, 64B line, 12/20-way, 16/48 MSHRs, LRU, 5/15-cycle round-trip latency
LLC	3MB/core, 64B line, 12-way, 64 MSHRs/slice, LRU, 55-cycle round-trip latency
(B) Apple A14 Firestorm (A14)	
Core	1 core, 8-wide fetch/execute/commit, 632-entry ROB, 148/106-entry LQ/SQ, perceptron branch predictor
L1/L2 Cache	Private, 128KB/4MB, 64B line, 8/16-way, 16/48 MSHRs, LRU, 3/16-cycle round-trip latency
LLC	Shared, 8MB, 64B line, 16-way, 64 MSHRs/core, LRU, 24-cycle round-trip latency
Main Memory (Both Architectures)	
GLC/A14	**1C**: 1 channel, 1 rank per channel;
GLC	**4C-16C**: 4 channels, 2 ranks per channel;
Both	8 banks per rank, DDR4-3200 MTPS, 64b data-bus per channel, 2KB row buffer, tRCD/tRP/tCAS=12.5ns

TABLE IV: Evaluated workloads.

Suite	Workload
SPEC 2006	mcf, milc, leslie3d, soplex, GemsFDTD, astar, wrf, sphinx3
SPEC 2017	bwaves, mcf, cactuBSSN, omnetpp, wrf, xalancbmk, fotonik3d, roms
GAP	bc (0, 3, 5, 12), bfs (3, 8, 10, 14), cc (5, 6, 13, 14), pr (3, 5, 10, 14), sssp (3, 5, 10, 14)

processor in 1-core and 2-core configurations. Table III(B) details its parameters.

Evaluated workloads. We evaluate MARS on 30 memory-intensive workloads drawn from SPEC CPU2006 [24] and SPEC CPU2017 [25], each exhibiting LLC MPKI greater than 1 in a baseline without prefetching or forwarding. Evaluations include homogeneous configurations with each core running the same trace and heterogeneous 4-core mixes with distinct traces per core. For every SPEC benchmark, the cache warms up over 50M instructions and statistics are collected over the next 200M instructions, matching standard convention [8], [19], [33], [34].

We additionally evaluate MARS on 20 GAP workloads [26], covering five graph algorithms: Betweenness Centrality (bc), Breadth First Search (bfs), Connected Components (cc), PageRank (pr), and Single Source Shortest Path (sssp). Traces are drawn from the ML-Based Data Prefetching Competition [35].

Compared mechanisms. We evaluate MARS across a three-axis mechanism matrix of off-chip prediction, L2 prefetcher, and replacement policy. Off-chip predictors: TTP [21], HMP [20], and Hermes [21]. Prefetchers deployed at L2: Bingo [9], Berti [8], SPP [10], and Zion [12]. Replacement policies: SHiP++ [17], Hawkeye [18], and CARE [19] deployed at LLC.

Metrics. We report normalized weighted speedup [34], [36] over a no-prefetch, LRU replacement baseline unless stated otherwise. MSHR admission stall reduction and pure miss reduction are measured relative to the TTP no-prefetch baseline, capturing the two mechanisms that underpin IPC gains.

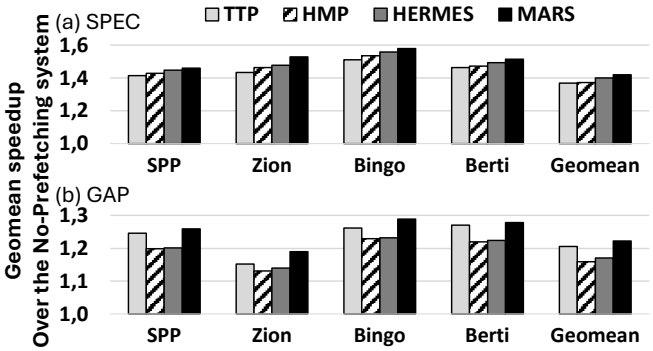

Fig. 5: Single-core geometric mean IPC speedup across SPEC and GAP workloads with and without L2 prefetching.

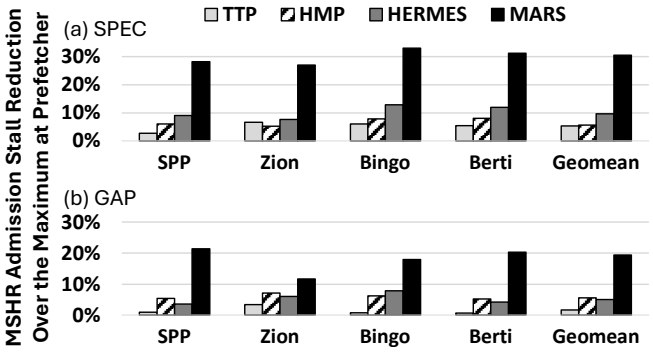

Fig. 6: L1 MSHR admission stall reduction relative to TTP no-prefetch baseline, single-core.

V. Results

A. Single-Core Performance Analysis

Figure 5(a) presents geometric mean IPC speedup in SPEC workloads under 1-core GLC configuration, using no-prefetch and four L2 prefetcher configurations (SPP, Zion, Bingo, and Berti). MARS achieves a geometric mean IPC speedup of 42.1%, outperforming TTP, HMP, and Hermes by 5.1%, 4.9%, and 2.1%, respectively. Figure 5(b) presents results in GAP workloads under the same configuration, where MARS (22.3%) outperforms TTP (20.6%), HMP (15.9%), and Hermes (17.1%), respectively. Figure 4 demonstrates that since MARS targets both MSHR admission stalls and history-discrepant misses that degrade per-level MHC, it consistently outperforms all off-chip prediction schemes across both workloads.

Off-chip predictors operate in parallel with the cache hierarchy and issue earlier DRAM requests affecting the return latency of on-chip requests. This cannot resolve the structural MHC bottlenecks of cache, regardless of which L2 prefetcher is being used.

Figures 6 and 7 present the fraction of L1 MSHR admission stalls and L1 pure misses eliminated by each scheme. On SPEC (Figures 6(a) and 7(a)), MARS achieves 35.7% MSHR admission stall reduction and 54.4% pure miss reduction in the geomean, while TTP, HMP, and Hermes eliminate 9.2%, 12.2%, and 11.8% of MSHR admission stalls and 35.0%, 32.4%, and 37.4% of pure misses, respectively. On GAP, with

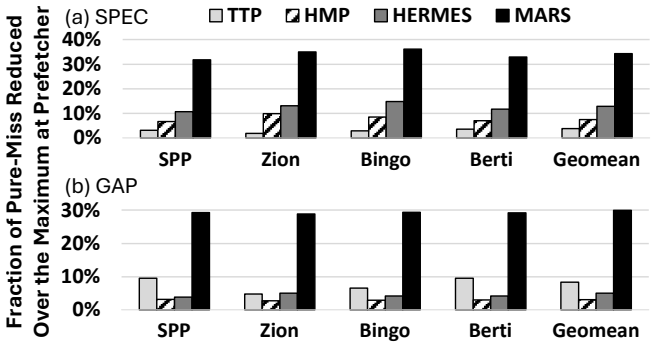

Fig. 7: L1 pure miss reduction relative to TTP no-prefetch baseline, single-core.

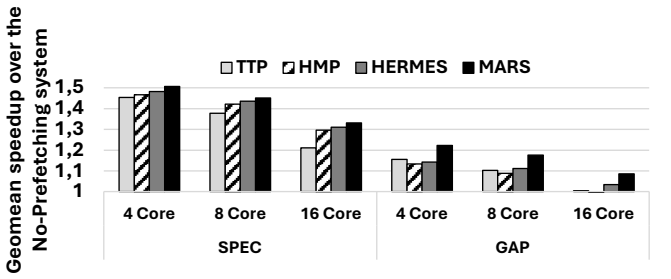

Fig. 8: Geometric mean IPC speedup for homogeneous 4-, 8-, and 16-core SPEC and GAP workloads.

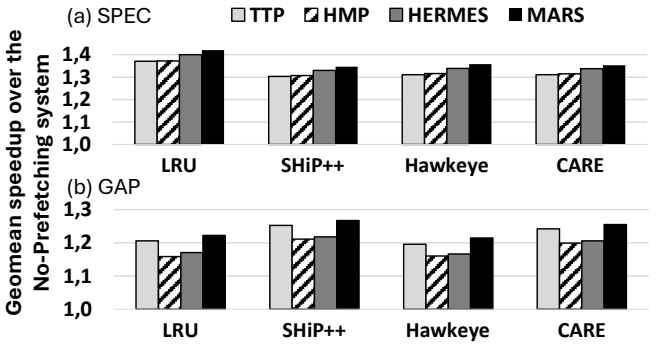

Fig. 9: Replacement policy sensitivity across LRU, SHiP++, Hawkeye, and CARE, single-core.

active L2 prefetching all mechanisms have worsened L1 MSHR admission stalls relative to the baseline, as inaccurate prefetcher requests generate additional MSHR traffic which exacerbates the MSHR admission stall at L1. MARS incurs the smallest degradation at 18.9%, compared to 33.3%, 37.6%, and 40.5% for TTP, HMP, and Hermes, respectively. However, the degradation only captures MSHR admission stalls, while overall pure miss reduction remains achievable. MARS eliminates 31.4%, compared to 12.3%, 4.5%, and 5.6% for TTP, HMP, and Hermes, respectively. The consistent pure miss reduction across both workloads indicates MARS effectively increases MHC in the memory system.

PALACE HOLDER

B. Multi-Core Performance Analysis

Figure 8 presents normalized weighted speedup across 4-core, 8-core, and 16-core homogeneous configurations, averaged over no-prefetch, SPP, Zion, Bingo, and Berti. On SPEC, MARS achieves ¡4C SPEC¿%, ¡8C SPEC¿%, and ¡16C SPEC¿% at 4, 8, and 16 cores, outperforming TTP, HMP, and Hermes across all configurations. On GAP, MARS achieves ¡4C GAP¿%, ¡8C GAP¿%, and ¡16C GAP¿% at 4, 8, and 16 cores. As core count scales, contention for shared LLC and MSHR resources intensifies. MARS sustains its advantage because its admission control operates per-level, targeting the structural MHC bottleneck that compounds underincreased thread pressure. Figure 11 presents normalized weighted speedup across 495 heterogeneous 4-core SPEC mixes sorted by speedup, prefetcher-averaged. MARS outperforms TTP, HMP, and Hermes across virtually every mix, with a geomean weighted speedup of ¡MARS hetero¿% compared to ¡TTP hetero¿%, ¡HMP hetero¿%, and ¡Hermes hetero¿%, respectively. The advantage is consistent across mix indices, indicating that MSHR admission stall reduction generalizes across workload interference patterns.

C. Performance Sensitivity Analysis

Figure 9(a) presents normalized weighted speedup under four LLC replacement policies (LRU, SHiP++, Hawkeye, and CARE) across SPEC workloads. MARS achieves ¡LRU¿%, ¡SHiP++¿%, ¡Hawkeye¿%, and ¡CARE¿% under LRU, SHiP++, Hawkeye, and CARE, respectively, outperforming TTP, HMP, and Hermes in all four configurations. Figure 9(b) presents GAP workloads under the same policies, where the same trend holds. This is because MARS targets L1 MSHR admission stalls, which are upstream of the LLC replacement decision, making its benefit orthogonal to the eviction policy.

Figure 10 presents normalized weighted speedup on the A14 architecture under 1-core and 2-core configurations across SPEC and GAP workloads. MARS achieves ¡1C SPEC A14¿% and ¡2C SPEC A14¿% on SPEC, and ¡1C GAP A14¿% and ¡2C GAP A14¿% on GAP, outperforming TTP, HMP, and Hermes across all configurations. These results confirm that MARS generalizes across microarchitectural generations and is not specific to the GLC pipeline.

REFINED RESUTLS PLACEHOLFDER

Takeaway. Across the eight result blocks, MARS delivers IPC gain on both SPEC and self-attention workloads at the LRU baseline (A, B), composes with all three evaluated replacement policies (C), outperforms off-chip prediction across replacement choices (D), scales to heterogeneous multi-core configurations (E), achieves non-zero L1 ASR where off-chip prediction yields zero (F), generalizes to the Apple A14 microarchitecture (G), and operates at sub-100-bit storage (H). The result set closes the contribution: selective per-level forwarding addresses the admission stall regime that prefetching, replacement, and off-chip prediction each leave unaddressed.

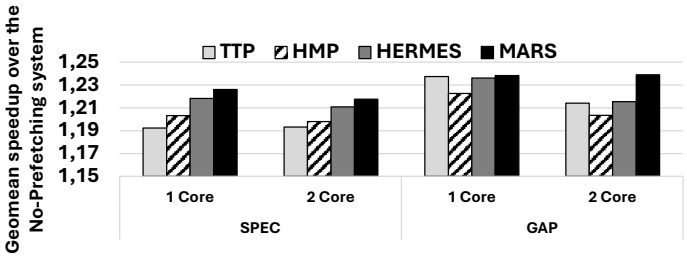

Fig. 10: Architecture sensitivity under A14 configuration, 1-core and 2-core SPEC and GAP workloads.

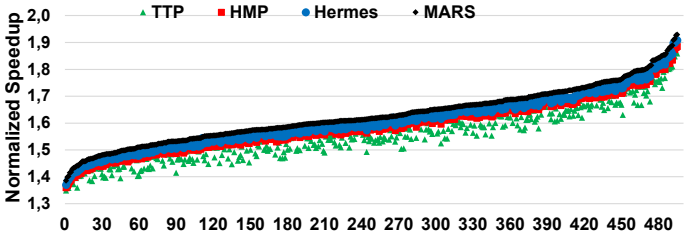

Fig. 11: Normalized weighted speedup across 495 heterogeneous 4-core SPEC workload mixes.

VI. Conclusion

This paper presents MARS, a selective, adaptive, and lightweight per-level forwarding framework targeting MSHR Admission stall cycles. A level enters the admission stall regime when its miss-handling capacity is saturated and cannot admit newly detected misses. Prior families leave this regime unaddressed. By forwarding qualifying misses to the next level, MARS extends the level's effective Miss Handling Concurrency past its structural bound while adding 92 bits of counter state per gated level (L1, L2) and a 3-bit skip tag per in-flight miss, with no MSHR entry allocated at any skipped level. The three-gate forwarding decision reads short- and long-windowed φ and κ signals derived from the concurrent memory access model, fires inline at miss-time, and composes with existing prefetchers and replacement policies without shared state. Across 30 SPEC CPU traces and 15 self-attention workloads, MARS reduces L1 MSHR Admission stall cycles and delivers consistent IPC gain, establishing selective forwarding as a distinct and lightweight mechanism slot in the cache hierarchy.

References

[1] W. A. Wulf et al., "Hitting the memory wall: Implications of the obvious," ACM SIGARCH computer architecture news, vol. 23, no. 1, pp. 20–24, 1995.

[2] X.-H. Sun and X. Lu, "The memory-bounded speedup model and its impacts in computing," Journal of Computer Science and Technology, vol. 38, no. 1, pp. 64–79, 2023.

[3] Y. Chou, B. Fahs, and S. Abraham, "Microarchitecture optimizations for exploiting memory-level parallelism," in Proceedings of the 31st Annual International Symposium on Computer Architecture (ISCA), 2004, pp. 76–87.

[4] X.-H. Sun and D. Wang, "Concurrent average memory access time," Computer, vol. 47, no. 5, pp. 74–80, 2014.

[5] D. Kroft, "Lockup-free instruction fetch/prefetch cache organization," in ISCA, 1998, pp. 195–201.

[6] S. Belayneh and D. R. Kaeli, "A discussion on non-blocking/lockup-free caches," SIGARCH Computer Architecture News, vol. 24, no. 3, pp. 18–25, 1996.

[7] M. K. Qureshi, D. N. Lynch, O. Mutlu, and Y. N. Patt, "A case for mlp-aware cache replacement," ACM SIGARCH Computer Architecture News, vol. 34, no. 2, pp. 167–178, 2006.

[8] Navarro-Torres et al., "Berti: an accurate local-delta data prefetcher," in MICRO, 2022, pp. 975–991.

[9] M. Bakhshalipour et al., "Bingo spatial data prefetcher," in HPCA, 2019, pp. 399–411.

[10] J. Kim et al., "Path confidence based lookahead prefetching," in MICRO, 2016, pp. 1–12.

[11] M. Shakerinava et al., "Multi-lookahead offset prefetching," DPC3, no. 2019, 2019.

[12] V. Biryukov, X. Lu, Z. Liu, K. Zhou, and X.-H. Sun, "Zion: A comprehensive, adaptive, and lightweight hardware prefetcher," in DATE, 2026.

[13] X. Lu et al., "Apac: An accurate and adaptive prefetch framework with concurrent memory access analysis," in ICCD, 2020, pp. 222–229.

[14] S. Srinath et al., "Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers," in HPCA. IEEE, 2007, pp. 63–74.

[15] A. Jaleel, K. B. Theobald, S. C. S. Jr., and J. Emer, "High performance cache replacement using re-reference interval prediction (RRIP)," in ISCA, 2010, pp. 60–71.

[16] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. S. Jr., and J. Emer, "SHiP: Signature-based hit predictor for high performance caching," in MICRO, 2011, pp. 430–441.

[17] C.-J. Wu, A. Jaleel, M. Martonosi, and S. C. S. Jr., "SHiP++: Enhancing signature-based hit predictor for high performance caching," 2016, 2nd Cache Replacement Championship (CRC2), ISCA.

[18] A. Jain and C. Lin, "Back to the future: Leveraging Belady's algorithm for improved cache replacement," in ISCA, 2016, pp. 78–89.

[19] X. Lu, R. Wang, and X.-H. Sun, "CARE: A concurrency-aware enhanced lightweight cache management framework," in HPCA. IEEE, 2023, pp. 1208–1220.

[20] A. Yoaz, M. Erez, R. Ronen, and S. Jourdan, "Speculation techniques for improving load related instruction scheduling," in ISCA, 1999, pp. 42–53.

[21] R. Bera et al., "Hermes: Accelerating long-latency load requests via perceptron-based off-chip load prediction," in MICRO, 2022, pp. 1–18.

[22] H. Geng, X. Lu, Y. Che, Z. Tian, D. Cheng, X.-H. Sun, M. Niemier, and X. S. Hu, "Cosmos: Rl-enhanced locality-aware counter cache optimization for secure memory," in Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture, 2025, pp. 1073–1086.

[23] J. Tuck, L. Ceze, and J. Torrellas, "Scalable cache miss handling for high memory-level parallelism," in MICRO, 2006.

[24] "SPEC CPU2006 Benchmark Suite," http://www.spec.org/cpu2006/, 2006.

[25] "SPEC CPU2017 Benchmark Suite," http://www.spec.org/cpu2017/, 2017.

[26] S. Beamer, K. Asanović, and D. Patterson, "The GAP benchmark suite," arXiv preprint arXiv:1508.03619, 2015.

[27] J. Liu, P. Espina, and X.-H. Sun, "A study on modeling and optimization of memory systems," Journal of Computer Science and Technology, vol. 36, no. 1, pp. 71–89, 2021.

[28] Y. Liu and X.-H. Sun, "Lpm: A systematic methodology for concurrent data access pattern optimization from a matching perspective," IEEE Transactions on Parallel and Distributed Systems, vol. 30, no. 11, pp. 2478–2493, 2019.

[29] N. Gober et al., "The Championship Simulator: Architectural Simulation for Education and Competition," 2022.

[30] "2nd cache replacement championship." https://crc2.ece.tamu.edu/, 2017.

[31] "3rd cache replacement championship." https://crc2.ece.tamu.edu/, 2019.

[32] E. Rotem et al., "Intel alder lake cpu architectures," MICRO, pp. 13–19, 2022.

[33] P. Papaphilippou et al., "Pangloss: a novel markov chain prefetcher," arXiv preprint arXiv:1906.00877, 2019.

[34] S. Pakalapati et al., "Bouquet of instruction pointers: Instruction pointer classifier-based spatial hardware prefetching," in ISCA, 2020, pp. 118–131.

[35] A. Jain et al., "Ml-based data prefetching competition," 2021, mLArchSys Workshop, ISCA 2021.

[36] X. Lu et al., "Chrome: Concurrency-aware holistic cache management framework with online reinforcement learning," in HPCA, 2024, pp. 1154–1167.